

Ocenjevanje rešitve Snoj–Zmazek

Maksimalno inducirano ujemanje — slabi vhodni podatki

1 Uvod

Ocenjujemo rešitev problema maksimalnega induciranege ujemanja (MIM). Rešitev sestavljata dve fazi: (1) požrešni algoritem z lokalnim iskanjem ((1,2)-zamenjave) in (2) algoritem razveji-in-omeji z začetno mejo iz prve faze. B&B se razvejuje na vozlišču z najmanjšo stopnjo, za rezanje uporablja zgornji meji $\lfloor \text{aktivna vozlišča} / 2 \rfloor$ in število preostalih povezav, pred razvejanjem pa odstrani izolirana vozlišča in vsiljuje izolirane povezave (oba krajišča stopnje 1).

Privzeta časovna omejitev je 300 s; velikost grafa je omejena z bitnim poljem na $n \leq \text{MAXN} = 1024$. V nadaljevanju predstavimo tri vhode, ki ciljajo na različne šibke točke. Meritve so opravljene s `./solver vhod izhod 5` (5 s omejitev) za hitro reprodukcijo.

2 Vhod 1: `bad_disjoint_triangles.in`

2.1 Konstrukcija

Disjunktna unija $k = 341$ trikotnikov K_3 , skupaj $n = 1023$ vozlišč in 1023 povezav. Vsa vozlišča imajo stopnjo 2.

2.2 Optimum

V vsakem K_3 izbira ene povezave izloči preostalo vozlišče, zato $\text{MIM} = k = 341$. To mejo doseže že požrešni algoritem.

2.3 Težavnost za algoritem

Rešitev je optimalna, vendar je B&B ne uspe potrditi. Po prvi fazi velja `bestSize` = k , zgornja meja na korenu pa znaša

$$\lfloor 3k/2 \rfloor = \lfloor 1,5 k \rfloor,$$

torej za faktor 1,5 ohlapnejša od optimuma. Pogoji `curEdges + ub` ≤ `bestSize` se sproži šele, ko iz aktivne množice odstranimo več kot k vozlišč. Redukciji ne delujeta: ni vozlišč stopnje 0 niti 1. Branching faktor je $\text{deg}_{\min} + 1 = 3$.

2.4 Meritev

```
./solver evaluation/data/bad_disjoint_triangles.in /tmp/out.txt 5
Solution size: 341, nodes: 2695168, time: 5.006s, timed_out: true
```

Algoritem vrne pravilno rešitev 341, vendar je `timed_out` = `true` tudi pri 60 s in privzetih 300 s.

3 Vhod 2: bad_random_3reg.in

3.1 Konstrukcija

Naključen 3-regularen graf na $n = 1024$ vozliščih, zgrajen z modelom konfiguracije. Velikost je največja, ki jo bitno polje ($\text{MAXN} = 1024$, $\text{nWords} = 16$) še sprejme.

3.2 Težavnost za algoritem

- Vsa vozlišča imajo stopnjo 3, zato nobena redukcija ne sproži.
- Začetna zgornja meja $\lfloor n/2 \rfloor = 512$ je daleč nad dejanskim optimumom (požrešnica vrne 271).
- Branching faktor je $\deg_{\min} + 1 = 4$.
- Pri $n = 1024$ pohitritev z bitnimi operacijami ne kompenzira eksponentne rasti drevesa.

3.3 Meritev

```
./solver evaluation/data/bad_random_3reg.in /tmp/out.txt 5
Solution size: 271, nodes: 1732608, time: 5.011s, timed_out: true

./solver evaluation/data/bad_random_3reg.in /tmp/out.txt 60
Solution size: 271, nodes: 20123648, time: 60.007s, timed_out: true
```

Pri 60 s se rešitev ne izboljša.

4 Vhod 3: bad_random_4reg.in

4.1 Konstrukcija

Naključen 4-regularen graf na $n = 500$ vozliščih s 1000 povezavami.

4.2 Težavnost za algoritem

Primer cilja predvsem na branching faktor: ker je $\deg_{\min} = 4$, se drevo razvejuje s 5 namesto 4 vejami. Hkrati nobena redukcija ne sproži, meja $\lfloor n/2 \rfloor = 250$ pa je daleč nad dejanskim optimumom (požrešnica ≈ 112).

4.3 Meritev

```
./solver evaluation/data/bad_random_4reg.in /tmp/out.txt 5
Solution size: 112, nodes: 3686400, time: 5.002s, timed_out: true

./solver evaluation/data/bad_random_4reg.in /tmp/out.txt 60
Solution size: 112, nodes: 40054784, time: 60.005s, timed_out: true
```

5 Pregled in zaključek

Datoteka	n	Vrnjeno	Čas/Status	Šibka točka
bad_disjoint_triangles.in	1023	341 (opt.)	5 s, TLE	ohlapna meja $ V /2$
bad_random_3reg.in	1024	271	5 s, TLE	globina drevesa, velikost
bad_random_4reg.in	500	112	5 s, TLE	branching faktor 5

Vsak vhod izpostavi drugo pomanjkljivost: pri disjunktnih trikotnikih ohlapno zgornjo mejo $|V|/2$, pri 3-regularnem grafu odsotnost redukcij ob največji dovoljeni velikosti, pri 4-regularnem grafu pa povečan branching faktor. V vseh primerih solver vrne rešitev z zastavico `timed_out = true` že pri 5 s. Pri trikotnikih je rešitev optimalna, a nedokazana; v drugih dveh primerih je verjetno suboptimalna.

Reprodukcija

```
go build -o solver solver.go
for f in evaluation/data/*.in; do
    ./solver "$f" /tmp/out.txt 5
done
```